

Esercizio 1 Lettore-pulsante-stampante

giovedì 5 marzo 2026 14:02

Un processore z64 controlla la cassa di un supermercato. Esso è collegato a tre periferiche: LETTORE, che legge i codici a barre degli articoli; PULSANTE, che informa lo z64 che si richiede la stampa dello scontrino; STAMPANTE, che effettua tale stampa. Ogni volta che un codice a barre viene letto, LETTORE invia un'interruzione allo z64. Il servizio associato all'interruzione di LETTORE consiste nelle seguenti operazioni:

- lo z64 legge dalla periferica il numero dell'articolo, a 16 bit;
- questo numero indica una posizione all'interno di un vettore di prezzi; ogni prezzo è una word e sono presenti 512 elementi nel prezzario.
- il prezzo viene copiato in coda alla lista dei prezzi degli articoli del cliente corrente. Questa lista è un vettore, la cui lunghezza massima è di 512 elementi. Quando viene premuto il bottone della periferica PULSANTE, questa invia un'interruzione allo z64. Il servizio associato a questa interruzione deve:
- inviare uno per volta (in busy waiting) i prezzi memorizzati fino a questo momento alla periferica STAMPANTE, che provvederà a stamparli su uno scontrino;
- inviare (in busy waiting) la somma dei prezzi a STAMPANTE, affinché la stampi;
- la lista dei prezzi deve essere svuotata, in modo che possa essere utilizzata per memorizzare i prezzi degli articoli del cliente successivo. In altre parole, quando si preme il pulsante, il prezzo del prossimo articolo letto (dopo la stampa dello scontrino) va nella posizione zero. Tutte le routine di servizio devono essere non interrompibili e durante il trasferimento dei dati a STAMPANTE la periferica LETTORE non deve poter leggere altri codici a barre.

Progettare e implementare:

- le interfacce delle periferiche STAMPANTE e LETTORE
- i driver per la gestione delle interruzioni inviate da LETTORE e PULSANTE

SUGGERIMENTO: è possibile utilizzare un contatore memorizzato in una variabile globale per tenere traccia del numero di articoli processati da LETTORE.

Soluzione

```
.org 0x800
.data:
    .equ $lettore_irq,0x00
    numero_articolo: word 0
    prezzi : .fill 512,0,16 #512 elementi di word
    nr_pezzi : .word 0
    .equ $pulsante_status , 0x01
    .equ $pulsante_reg,0x02
    articoli_cliente : fill 512,,0,16 #tutti i prezzi del cliente
    prezzo_totale : . word 0
    prezzo_articolo: .word 0

.text
main:
    Sti
    .loop:
        Hlt
        Jmp loop

.driver 0: #lettore
    Push %rax
    Movb $0,%al #disabilito interruzioni
    Outw %al,lettore_irq
    Call leggi
    Pop %rax
    Iret

Leggi:
    Pushq %rax
    Pushq %rcx
    #leggo codice
    Inw %ax, numero_articolo
    Xorw %cx, %cx #azzerò rcx
    .trova_prezzo:
        Cmpw numero_articolo, prezzi(,%cx,2) #vedo se c'è il codice
        Jnz .incrementa #prezzo non c'è a indice %rcx
    .incrementa:
        Addb $1,%cx
        Movw prezzi(,%cx,2),prezzo_articolo
        Addw $1,nr_pezzi #nr_pezzi++
        #vedo se nr_pezzi < 512
        Cmpw $512,nr_pezzi
        Jnz .trova_prezzo
```

```

Pop %rcx
Pop %rax
Ret
.driver 1 : #pulsante
Push %rax
Outb %al,$pulsante_status #azzero status
Outb %al,$pulsante_reg
Call scontrino
Pop %rax
Iret
Scontrino:
Push %rax
Push %rcx
Outb %al,$pulsante_status #status=0
Xorw %cx,%cx #azzero %rcx
.loop:
Movw articoli_cliente(,%cx,2),%ax
Outw %ax,$pulsante_reg #c'è il prezzo
.bw:
Inb $pulsante_status,%al
Btb $0,%al
Jnc .bw

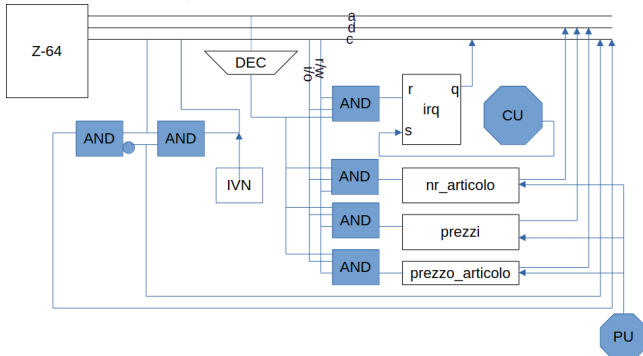
#dato pronto
Addw %ax,prezzo_totale
Xorq %al,%al #azzero al
Outb %al,$pulsante_status
.bw1:
Inb $pulsante_status,%al
Btb $0,%al
Jnc .bw1

Movw $prezzo_totale,%ax
Outw %ax,$pulsante_reg
#uso 0 per fine
Cmpw $0,%cx(articoli_cliente)
Jz .exit
Jnz .continue

.exit:
Xorq %rcx,%rcx #azzero contatore
Popq %rcx
Popq %rax
Ret
.continue:
#diverso da 0
Addw $1,%cx #%%cx ++
Jmp .loop

```

Schema per lettore (irq)



Schema per stampante (bw)

